Polytech Dijon

5A Informatique et Electronique

Projet Embedded Systems and Interfaces

Video Streaming

Done by :

FONKOUA CHARLE LUCE SORIANE
MAMBO MOTSOU JUNIOR D.

Supervised by :

Mr. Julien DUBOIS

Year 2025–2026

Sommaire

1 Introduction **4**

2 Project Objectives **5**

3 Theoretical Background **6**

3.1 What is an image? 6
3.2 Video timing principles 6
3.2.1 Active video 6
3.2.2 Blanking periods 6
3.2.3 Sync signals (hsync, vsync) 6
3.2.4 Front porch / back porch 7
3.3 VGA signaling and constraints 7
3.4 AXI4-Stream protocol basics 7
3.5 High Level Synthesis (HLS) principles 7
3.6 FPGA debugging principles (ILA, simulation) . . . 7
3.7 Video timing principles in practice 7

4 Basic VGA Video Chain **8**

4.1 Pixel clock generation using the Clock Wizard . . . 8
4.2 Implementation of the 8-bit binary counter 9
4.3 Synchronization using the Video Timing Controller (VTC) 10
4.4 Explanation of blanking periods, appearance of black bands, and simulation results 10
4.5 Bit extraction module 11
4.6 Hardware debugging using the Integrated Logic Analyzer (ILA) 12

5 AXI4-Stream Video Pipeline **14**
5.1 Conversion from video to AXI using the "Video In to AXI-Stream" module 14
5.2 Generation of output timing with the second Video Timing Controller (VTC) 15
5.3 Conversion back to video using the "AXI-Stream to Video Out" module 15
5.4 Final block architecture of the AXI4-Stream video pipeline 16

6 Video Processing Modules **18**
6.1 Manual VHDL-based grayscale inversion 18
6.2 Integration of the inversion module into the AXI stream 18
6.3 High-Level Synthesis (HLS) module for grayscale inversion 19
6.3.1 Video_invert IP: Function and Integration . 19
6.4 HLS module for overlaying the binôme number . . . 20
6.4.1 Video incrustation IP: Function and Operation 20
6.5 System integration and final block architecture . . . 22
6.6 Test and Validation 23

7 Problems and Solutions **25**

8 Conclusion **26**

A appendix **27**

Table des Figures

4.1 Clock Wizard configuration for 108 MHz system clock and 25.2 MHz pixel clock 8
4.2 binary counter 9
4.3 blanking periods, appearance of black bands, and simulation results 11
4.4 Bit extraction module 11
4.5 VHDL bit extraction test 12
4.6 Vsync 13
4.7 ILA debugger connection 13

5.1 Video in to Axi4 stream IP 15
5.2 Axi4 stream in to video IP 16
5.3 AXI4-Stream video pipeline 17

6.1 Complete video chain system 19
6.2 HLS and VHDL synthesized video inverter 20
6.3 Hls synthesized video incrustation 22
6.4 Complete chain with video incrustation 22
6.5 complete chain with video inverter 22
6.6 TB video inverter VHDL 24

A.1 Bit extraction test to show Vsync and Hsync synchronisation 27
A.2 Test showing each colour synchronisation 27
A.3 Bit extraction VHDL code 28

Chapter 1

Introduction

Video processing has become an essential component in modern electronic systems, ranging from embedded displays to advanced computer vision applications. Implementing a complete video chain on an FPGA provides a concrete way to understand how video signals are generated, synchronized, transmitted, and transformed in real time. Unlike software simulations, hardware implementation forces the designer to consider the strict timing constraints, clock domains, and interface protocols required to drive a physical display correctly.

In this practical , we explored the structure of a full video pipeline using the Zedboard development kit. The work began with the generation of a simple grayscale video pattern, allowing us to set up the VGA timing signals and validate the base video flow. Once the foundation chain was operational, the system was extended using AXI4-Stream interfaces, allowing the insertion of customized image-processing modules. This gave us the opportunity to design our own video operators both manually in VHDL and automatically through High Level Synthesis (HLS), and to integrate them seamlessly within the streaming architecture.

Chapter 2

Project Objectives

The main objective of this project was to develop a complete real-time video transmission system on an FPGA platform. Beyond achieving a functional implementation, the project aimed to provide a deeper understanding of the fundamental mechanisms involved in embedded video processing.

Several technical and educational objectives were pursued:

- **Understanding real-time video transmission principles**: analysis of the video pipeline, data flow management, latency constraints, and signal synchronization.
- **Mastering AXI protocols**: use of AXI4-Stream and AXI-Lite interfaces to enable communication between hardware blocks and ensure compatibility with the Zynq architecture.
- **Hardware and algorithmic development**: design of dedicated hardware modules, implementation of video processing algorithms using HLS, and integration of custom IP blocks.
- **FPGA integration**: assembly of all system components (DMA, SmartConnect, FIFO, video modules, Processor reset) within the Vivado Block Design environment.
- **Advanced hardware debugging**: use of the Integrated Logic Analyzer (ILA) to monitor internal signals, inspect video frames, and troubleshoot system malfunctions.
- **System-level approach**: gaining a global understanding of the entire video chain—from acquisition and processing to display—within the constraints of an embedded system.

In general, this practical session provided not only a functional video streaming system but also a comprehensive and realistic introduction to FPGA digital video processing, combining digital design, AXI communication, HLS development, and professional debugging methodology.

Chapter 3

Theoretical Background

3.1 What is an image?

An image is a two-dimensional array of pixels, where each pixel represents a color or intensity value. In digital systems, pixel values are encoded in binary, typically using 8 bits per channel (RGB), allowing for 256 intensity levels per color.

3.2 Video timing principles

Video timing defines how pixel data is transmitted to a display, ensuring proper synchronization between the source and the monitor. Timing signals control when pixels are visible and when blanking occurs.

3.2.1 Active video

Active video corresponds to the portion of each frame where pixel data is visible on the screen. During this period, RGB signals carry meaningful color information.

3.2.2 Blanking periods

Blanking periods are intervals during which no pixel data is transmitted. They occur at the end of each line (horizontal blanking) and at the end of each frame (vertical blanking) to allow the display to reposition the electron beam (CRT) or refresh logic (LCD).

3.2.3 Sync signals (hsync, vsync)

Horizontal sync (`hsync`) and vertical sync (`vsync`) signals indicate the start of a new line or frame, respectively. These signals are essential for the monitor to align and display pixel data correctly.

3.2.4 Front porch / back porch

Front porch and back porch are short intervals before and after sync pulses. They allow the display circuitry to stabilize and ensure that the visible portion of the frame is rendered correctly.

3.3 VGA signaling and constraints

VGA signaling consists of analog RGB signals accompanied by `hsync` and `vsync`. Digital implementations often convert these to discrete levels. Timing constraints must be respected precisely to avoid visual artifacts such as tearing or flickering.

3.4 AXI4-Stream protocol basics

AXI4-Stream is a widely used FPGA protocol for streaming data between modules. It supports continuous pixel flow with optional control signals such as `tvalid`, `tready`, and `tlast`, enabling efficient video processing pipelines.

3.5 High Level Synthesis (HLS) principles

HLS translates high-level code (C/C++) into RTL (VHDL/Verilog) for FPGA implementation. It allows rapid design exploration, supports pipelining, and facilitates integration with AXI4 interfaces for video streaming.

3.6 FPGA debugging principles (ILA, simulation)

Internal Logic Analyzers (ILA) and simulations allow observation of signals in real time or pre-silicon. They are essential to verify correct timing, synchronization, and data integrity in video streaming designs.

3.7 Video timing principles in practice

In a VHDL video streaming project, the pixel counter generates RGB values, while the Video Timing Controller generates `active_video`, `hsync`, and `vsync`. Gating the RGB output with `active_video` ensures black blanking periods, and slicing the most significant bits reduces grayscale levels, simplifying design without introducing artifacts.

Chapter 4

Basic VGA Video Chain

4.1 Pixel clock generation using the Clock Wizard

The first step in building the VGA output system was to generate the appropriate pixel clock. VGA displays require precise and stable frequencies to interpret pixel data correctly, and even slight deviations can result in distorted or unstable images. For the standard 640×480 resolution, the required pixel clock is close to 25 MHz. To meet this requirement, the Clock Wizard was configured to produce two distinct outputs:

- **108 MHz system clock**: used for internal logic and AXI-related operations.
- **25.2 MHz pixel clock**: dedicated to the VGA stream, ensuring correct synchronization with the display.

The 25.2 MHz clock drives both the pixel counter and the Video Timing Controller (VTC), ensuring that pixel generation aligns perfectly with the required timing signals. By stabilizing these two clock domains from the beginning, a reliable and consistent foundation was established for the entire video chain.

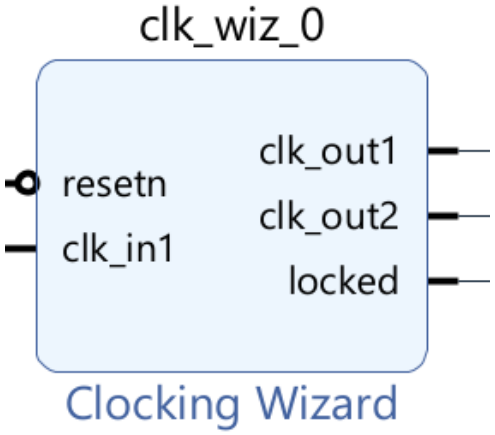


Figure 4.1: Clock Wizard configuration for 108 MHz system clock and 25.2 MHz pixel clock

4.2 Implementation of the 8-bit binary counter

To generate a simple and easily recognizable video pattern, we implemented an 8-bit binary counter.

This counter increments at every clock cycle of the 25.2 MHz pixel clock, producing a continuous sequence of values ranging from 0 to 255. When these values are interpreted as grayscale intensities, they form a horizontal gradient that spans the width of the screen.

To ensure that this pattern corresponds precisely to the visible region of the display, two control mechanisms were used:

- The counter increments only when the ***active*** video signal is asserted, ensuring that the pixel values evolve exclusively during the visible portion of each line.
- The counter is synchronously reset at each horizontal synchronization pulse (**hsync**), guaranteeing that each line begins with the same initial intensity.

Together, these two mechanisms ensure that the generated pattern remains stable, structured, and perfectly repeatable from frame to frame.

To control visual output and eliminate artifacts during blanking periods, a `Slice` component was used to select the four most significant bits (MSBs) of the 8-bit counter, effectively reducing grayscale precision to 16 levels while maintaining a clean visual gradient. However, extracting only the MSBs is insufficient to guaranty correct display: during blanking intervals, the monitor expects the RGB signals to be zero, but the counter continues running.

To address this, the sliced output is gated with the `active_video` signal from the Video Timing Controller. When `active_video` is high, the four MSBs are forwarded to the RGB outputs; when low, the output is forced to zero, ensuring black blanking periods.

This approach combines hardware slicing with active-video gating, confining the grayscale pattern strictly to the visible region, eliminating black bands caused by uncontrolled blanking, and producing a cleaner, more professional video output. It also follows good design practice by respecting video timing constraints and minimizing unnecessary transitions during non-visible periods.

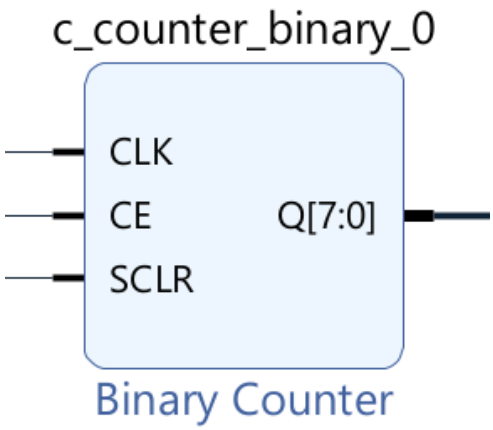


Figure 4.2: binary counter

4.3 Synchronization using the Video Timing Controller (VTC)

The Video Timing Controller (VTC) plays a central role in defining how the monitor interprets the incoming pixel data. VGA displays require not only pixel values, but also a set of synchronization signals indicating when each line and each frame begin. The VTC generates these signals according to the 640×480 VGA standard, including horizontal synchronization (**hsync**), vertical synchronization (**vsync**), and the **active video** interval.

To summarize its functions more clearly, the VTC:

- Generates all required VGA synchronization signals (**hsync**, **vsync**, blanking intervals).
- Enforces the correct timing for visible regions, front porch, sync pulses, and back porch.
- Drives the control signals used by the pixel counter and the gating logic for RGB outputs.
- Ensures perfect alignment between pixel generation and the expected display timing.

By providing these signals, the VTC guarantees that the system respects the temporal structure required by VGA displays. Without this synchronization, the monitor would not be able to reconstruct the image correctly.

4.4 Explanation of blanking periods, appearance of black bands, and simulation results

During the initial hardware tests, black bands appeared at the edges of the displayed image. These artifacts are directly linked to the blanking periods defined by the VGA standard. Key points to understand this behavior are:

- A VGA frame includes substantial intervals of time—both horizontally and vertically—during which no valid pixel data should be displayed.
- These intervals are known as the **horizontal** and **vertical blanking periods**, and they allow the monitor to reposition its scanning beam and prepare for the next line or frame.
- If the grayscale ramp generated by the counter continues during these blanking periods, the monitor receives pixel values when it expects none.
- Consequently, the display may interpret these out-of-spec signals as black bands or unstable brightness transitions.
- This phenomenon is not a design flaw but an inherent characteristic of how VGA video transmission operates.



Figure 4.3: blanking periods, appearance of black bands, and simulation results

The effectiveness was confirmed through simulation. The waveform clearly showed that the module output remained constant at zero during blanking periods, while the counter continued running internally.

As soon as active-video returned high, the module resumed outputting the correct grayscale value starting from the beginning of the ramp. The VHDL implementation and the corresponding simulation traces are provided in the annex Appendix **??**.

4.5 Bit extraction module

The initial version of the design relied on a simple `Slice` component to extract the four most significant bits of the 8-bit counter and send them directly to the VGA output. Key observations are:

- This approach successfully generated a basic grayscale pattern.
- However, it did not correct the behavior observed during blanking periods.
- The `Slice` has no awareness of video timing signals and continuously forwards the selected bits, even when **active-video** is low.
- As a result, the monitor continued receiving varying grayscale values during the blanking intervals.
- This caused the visible gradient to restart at each new line, appearing as multiple repeated **white-to-black** ramps on the screen.

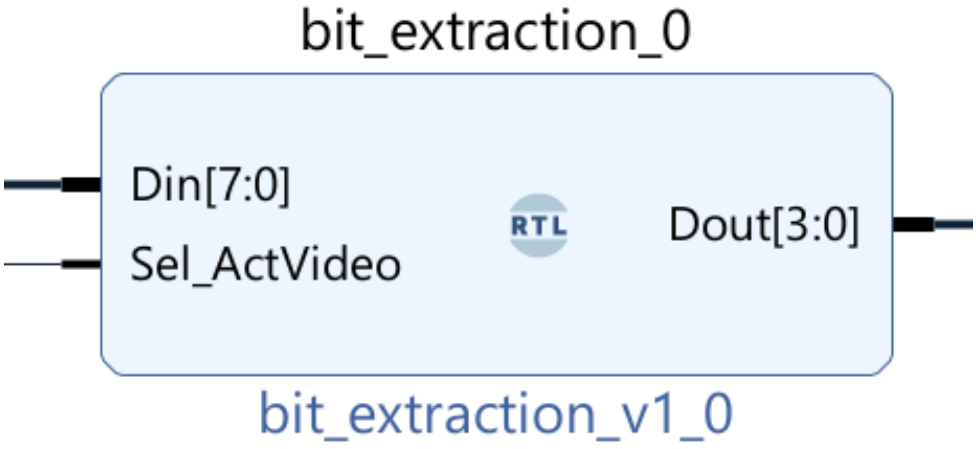


Figure 4.4: Bit extraction module

To address this problem, the Slice was replaced with a small custom bit-extraction module written in VHDL (see Appendix **??**). Key features of this module are:

- It performs the same bit selection as the **Slice**, extracting the 4 MSBs of the counter.
- It adds an essential control mechanism driven by the **active-video** signal.
- When **active-video** is high, the module outputs the extracted grayscale value normally.
- When **active-video** is low, the module forces the RGB outputs to zero.
- This ensures that the monitor receives valid pixel data only during the visible interval.
- All blanking periods are transmitted as pure black, exactly as required by the VGA protocol.

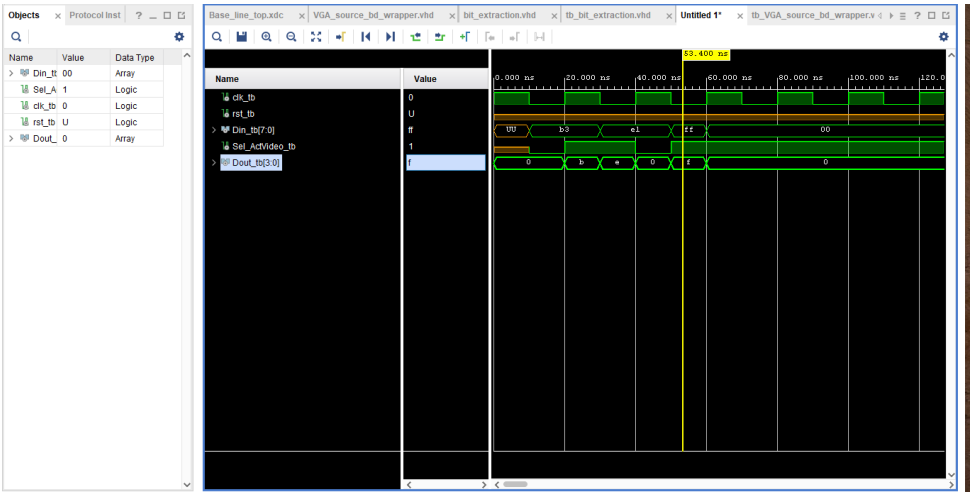


Figure 4.5: VHDL bit extraction test

4.6 Hardware debugging using the Integrated Logic Analyzer (ILA)

After validating the behavior of the video chain in simulation, the next step was to confirm that the design behaved identically on the actual FPGA. For this purpose, we used the Integrated Logic Analyzer (ILA), a built-in Xilinx debugging tool that allows internal signals to be sampled in real time while the design is running on hardware.

By inserting an ILA core directly into the block design and connecting it to key signals such as **hsync**, **vsync**, **active-video**, and the output of the bit-extraction module, we were able to monitor the precise timing relationships inside the physical implementation.

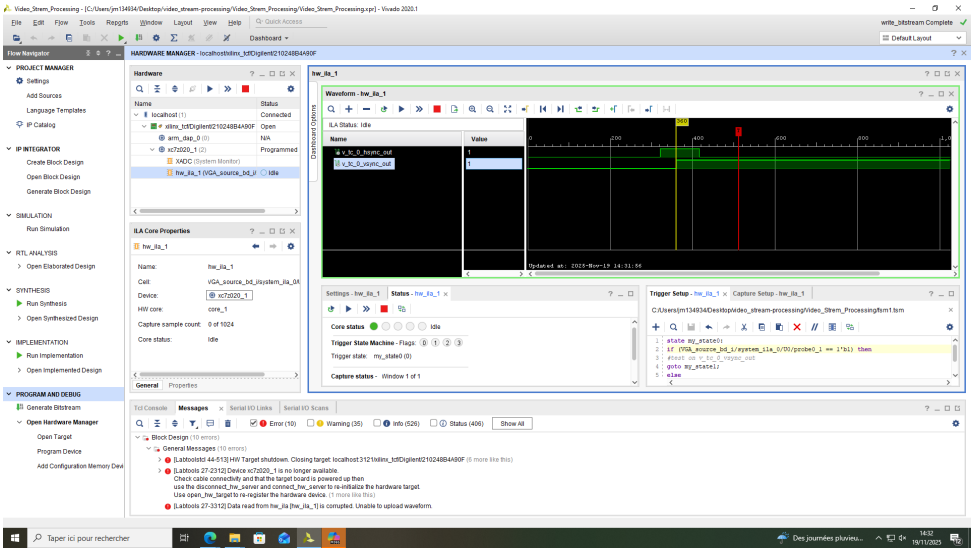


Figure 4.6: Vsync

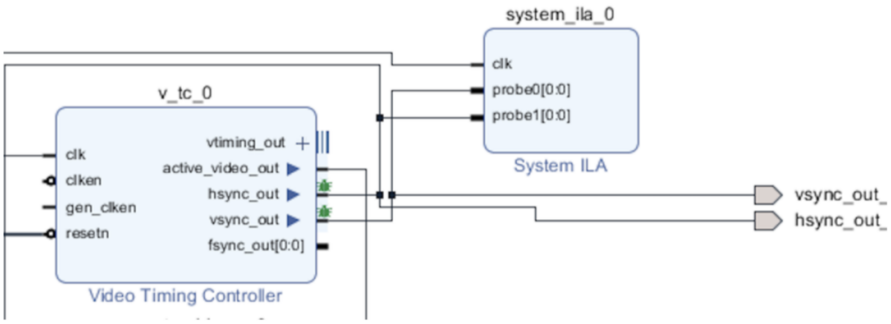


Figure 4.7: ILA debugger connection

Chapter 5

AXI4-Stream Video Pipeline

While the first part of the project established a functional VGA video chain, its structure was rigid: the pixel values were generated directly in hardware and flowed straight to the display with no flexibility for intermediate processing. To build a more modular and scalable system, we introduced the AXI4-Stream protocol, which is widely used in modern FPGA designs for high-throughput data transfer.

5.1 Conversion from video to AXI using the "Video In to AXI-Stream" module

The transition to an AXI-based architecture began with the "Video In to AXI-Stream" IP core. Made as follow:

- The module takes pixel data and timing signals produced by the Video Timing Controller and encapsulates them into the AXI4-Stream format.
- It generates the standard AXI signals: `tvalid`, `tready`, `tdata`, `tuser`, and `tlast`.
- The AXI stream operates at a higher frequency than the pixel clock, so the module uses independent clocks: 25.2 MHz for incoming video and 108 MHz for the AXI domain.
- A FIFO inside the core manages the timing differences between the two domains, preventing dropped or misaligned data.
- Converting the grayscale pixel stream into AXI format provides flexibility for inserting custom processing blocks later.

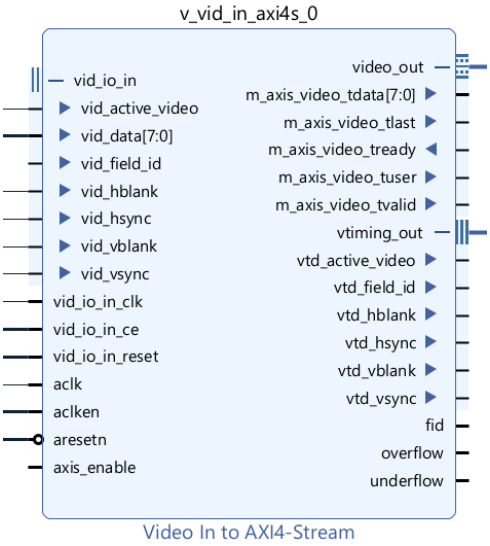


Figure 5.1: Video in to Axi4 stream IP

5.2 Generation of output timing with the second Video Timing Controller (VTC)

Once the video stream had been transformed into an AXI sequence, it became necessary to reconstruct the external synchronization signals for the display. For this purpose, we added a second Video Timing Controller, configured to detect the incoming timing metadata from the AXI stream and, based on that information, regenerate the hsync, vsync, and active video signals required by the VGA output.

This second VTC essentially acts as the "frame manager" on the output side, ensuring that the final processed video stream aligns perfectly with the expected VGA format. The module was configured in auto-generation mode for a 640×480 resolution, simplifying its integration into the design.

5.3 Conversion back to video using the "AXI-Stream to Video Out" module

To display the processed video on the VGA monitor, the AXI stream must eventually be converted back into standard pixel and synchronization signals. This role is fulfilled by the "AXI-Stream to Video Out" IP core. The module reads the incoming AXI packets, interprets the tuser and tlast markers to determine frame and line boundaries, and outputs the corresponding pixel values for each visible region.

The module was configured in 8-bit monochrome mode, consistent with the grayscale signal used throughout the project. Like the input module, it also uses independent clocks to bridge the 108 MHz AXI domain and the 25.2 MHz pixel domain. The correct operation of this component is crucial, as it must ensure that the processed pixel data aligns perfectly with the video timing generated by the second VTC.

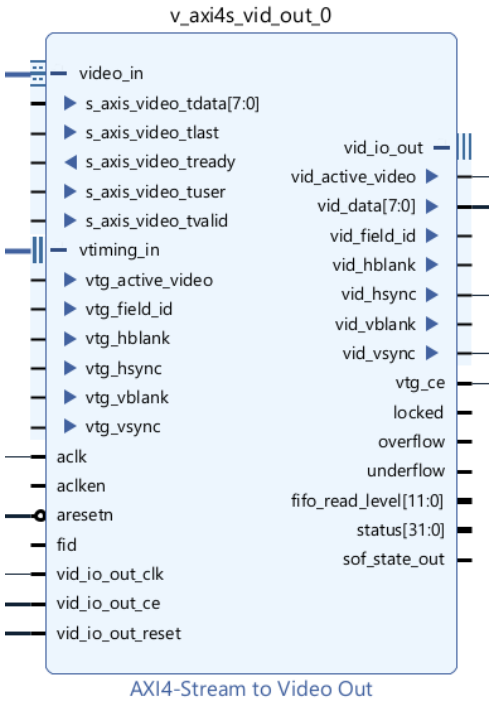


Figure 5.2: Axi4 stream in to video IP

5.4 Final block architecture of the AXI4-Stream video pipeline

To visualize how all the modules in this part interact, the complete AXI4-Stream video pipeline is summarized in the final block diagram shown below.

This diagram brings together the two Video Timing Controllers, the “Video In to AXI-Stream” and “AXI-Stream to Video Out” conversion modules, as well as the AXI interface signals and the internal FIFOs used for clock-domain crossing.

It provides a clear overview of the data flow from the VGA timing generation on the input side to the reconstruction of the processed video stream on the output side.

The diagram also highlights the position where the custom processing modules will later be inserted in Part 3, showing how the AXI stream allows us to modularly extend the video chain without modifying its core structure.

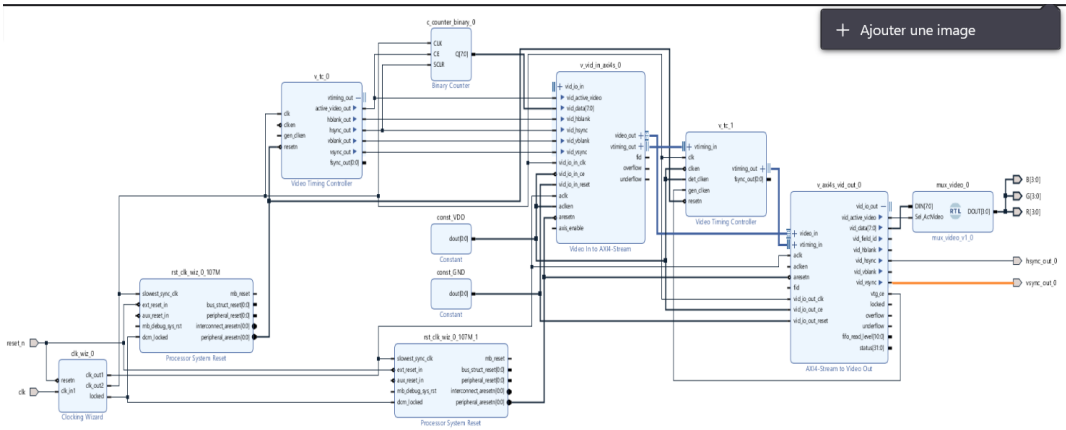


Figure 5.3: AXI4-Stream video pipeline

Chapter 6

Video Processing Modules

The first processing module implemented in the AXI4-Stream pipeline was a grayscale inversion block designed manually in VHDL. The objective of this module was straightforward: take each incoming 8-bit grayscale pixel and output its inverted value using the formula **pixel-out = 255 - pixel-in**.

6.1 Manual VHDL-based grayscale inversion

Although conceptually simple, implementing this operation within the AXI4-Stream framework required careful handling of the streaming interface signals, including tvalid, tready, tdata, tuser, and tlast.

The module was designed as an AXI slave on its input side and an AXI master on its output side, ensuring that it could receive data from the "Video In to AXI-Stream" module and pass its processed output directly to the "AXI-Stream to Video Out" module.

The completed VHDL implementation demonstrated the core principles of designing custom hardware operators within an AXI stream and served as the foundation for the more advanced HLS-based modules that followed.

6.2 Integration of the inversion module into the AXI stream

Once the inversion module was validated in simulation, it was inserted into the AXI pipeline between the input and output video conversion modules. This position allowed it to process each pixel between the moment it entered the AXI domain and the moment it was reconstructed back into VGA format.

The integration process required ensuring that the module properly propagated AXI flow-control signals so that it would neither stall nor lose data. When integrated, the module behaved as expected: each pixel was inverted in real time, and the resulting image displayed a clear visual reversal of the original grayscale gradient.

This successful integration confirmed that the pipeline could support custom processing blocks without compromising timing or synchronization.

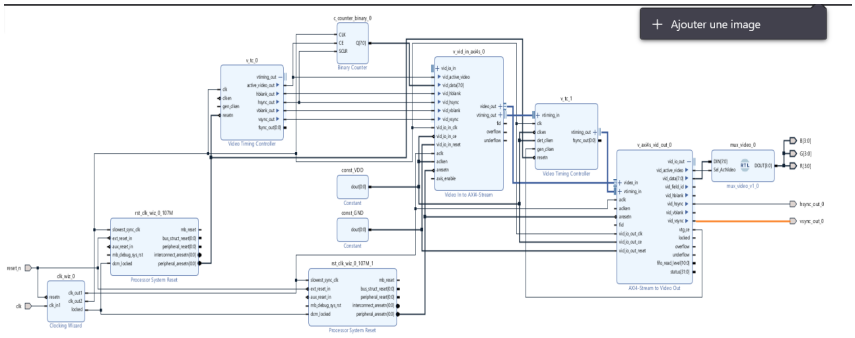


Figure 6.1: Complete video chain system

6.3 High-Level Synthesis (HLS) module for grayscale inversion

6.3.1 Video_invert IP: Function and Integration

The `Video_invert` IP is a simple AXI4-Stream video processing block that performs pixel inversion on an 8-bit grayscale stream. When the control signal `enable` is set to 1, each pixel value p is transformed into $255 - p$, producing a negative-video effect. When `enable` is 0, the video stream passes through unchanged.

How the IP Works

- The module receives video data through an AXI4-Stream interface (`axis_in`).
- For every clock cycle, one packet is read, containing:
  - `data`: 8-bit grayscale pixel value,
  - `tuser`: start-of-frame indicator,
  - `tlast`: end-of-line indicator,
  - `id`, `dest`: routing metadata.
- These side-channel signals are forwarded unchanged to the output.
- If `enable`=1, the output pixel becomes $255 -$ `data`. Otherwise, the pixel value is copied directly.
- The processed packet is written to the output AXI stream (`axis_out`).

The design is fully pipelined (`II=1`), allowing one pixel to be processed per clock cycle.

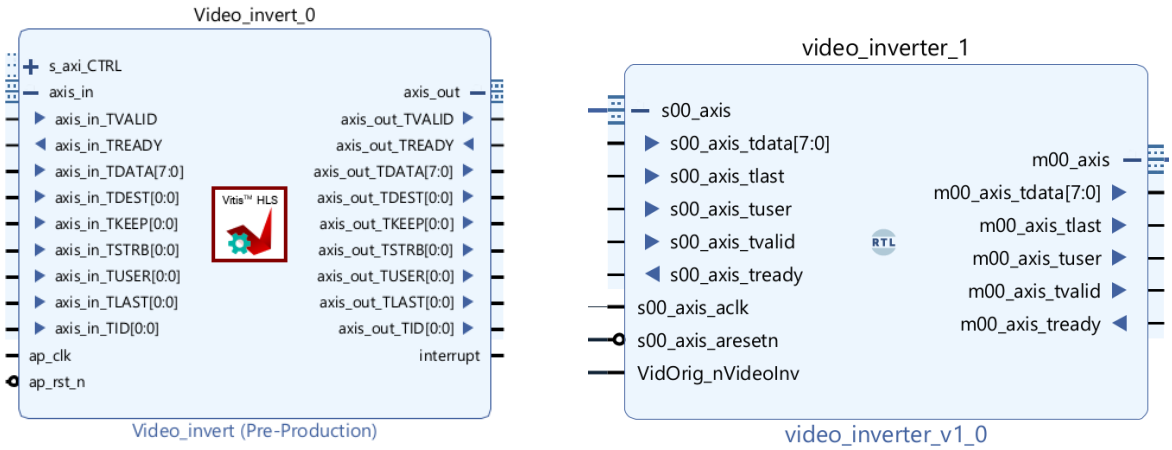
Role in the Video Chain

Within the video processing chain, the `Video_invert` IP is inserted between the "Video In to AXI4-Stream" block and the "AXI4-Stream to Video Out" block. Its function is to modify the pixel stream while preserving the video timing information carried by `tuser` (start of frame) and `tlast` (end of line).

The chain operates as follows:

1. The “Video In to AXI4-Stream” module converts the raw video timing signals into an AXI4-Stream of pixels.
2. The stream is fed into the `Video_invert` IP, which optionally inverts each grayscale pixel.
3. The processed stream is then passed to the “AXI4-Stream to Video Out” block, which regenerates the synchronization signals (`hsync`, `vsync`) and outputs a valid video stream.

This placement ensures that the inversion effect is applied in real time without modifying the structure of the video stream or the frame timing.



(a) HLS synthesized video inverter (b) VHDL synthesized video inverter

Figure 6.2: HLS and VHDL synthesized video inverter

6.4 HLS module for overlaying the binôme number

6.4.1 Video incrustation IP: Function and Operation

The `Video incrustation` IP overlays a two-digit number (0–9) on an incoming AXI4-Stream grayscale video. The number is drawn using a 5×7 monospace font and displayed starting at pixel position $(10, 10)$. When the `enable` register is set to 0, the module leaves the input stream unchanged.

Internal Operation

Each incoming AXI4-Stream word contains:

- `data`: an 8-bit grayscale pixel,
- `user`: asserted on the first pixel of the frame (Start of Frame),
- `last`: asserted on the last pixel of each line (End of Line),

- `id`, `dest`: side-channel routing information.

The module reconstructs pixel coordinates (x, y) from these control signals:

- When `tuser`=1, the module resets $(x, y) = (0, 0)$,
- When `tlast`=1, the line ends: $x = 0$, $y = y + 1$,
- Otherwise, $x = x + 1$ for each pixel.

The user-supplied 8-bit value `binome` is split into its tens and units digits. Each digit is rendered using a 5×7 bitmap stored in the `font5x7` table.

For each pixel, if `enable`=1:

- The module checks whether (x, y) lies inside the first digit area or the second digit area.
- If inside, it derives the font coordinates (f_x, f_y) and reads the corresponding bit in the font table.
- If the bit is equal to 1, the output pixel is forced to white (255).
- Otherwise, the original pixel value is forwarded.

If `enable`=0, the input pixel is copied exactly. All AXI4-Stream side-channel signals (`user`, `last`, `id`, `dest`) are propagated unchanged to preserve frame structure and timing.

Role in the Video Processing Chain

The `Video incrustation` module is inserted between two AXI4-Stream video IPs, as illustrated in the system diagram:

1. The "Video In to AXI4-Stream" block converts incoming video timing signals into an AXI4-Stream of pixels with valid `tuser` and `tlast`.
2. The stream is then processed by the `Video incrustation` IP, which optionally draws two overlay digits while maintaining one-pixel-per-cycle flow.
3. The processed stream is passed to "AXI4-Stream to Video Out", which regenerates VGA synchronization signals (`hsync`, `vsync`) and produces a standard video output.

Because the IP respects all AXI4-Stream video conventions and keeps the frame timing signals intact, the integration is fully compatible with the rest of the video chain, allowing real-time overlay at pixel clock speed (pipeline II=1).

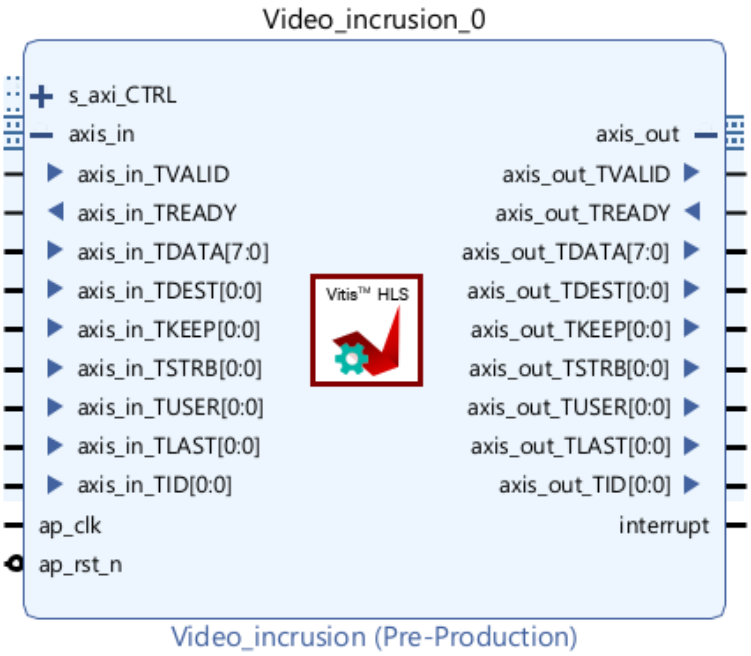


Figure 6.3: Hls synthesized video incrustation

6.5 System integration and final block architecture

After insertion of our IPs in the systems we obtain a complete functional video streaming system intended to perform video inversion and incrustation. They are shown in the figures below:

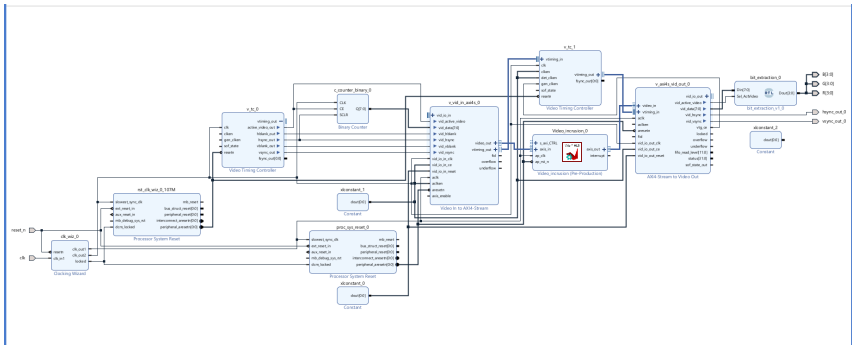


Figure 6.4: Complete chain with video incrustation

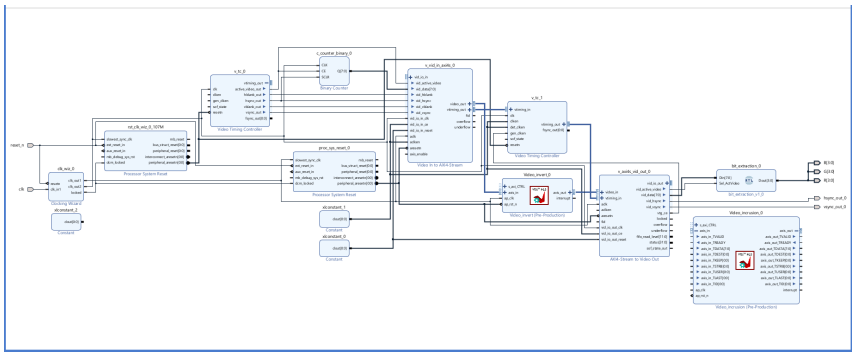


Figure 6.5: complete chain with video inverter

6.6 Test and Validation

After the IPs had been integrated to the chain they were tested but due to versioning issues the results wasn't visible because the Video upgrade of the video timing controller. A test bench was done with the VHDL version of the video inverter and we can perceive that the results as shown below:

Clock and Reset A 100 MHz clock drives the DUT. The reset signal (`s00_axis_aresetn`) is asserted low for the first 50 ns. After release, the internal registers become active.

Test 1: Inversion Enabled From 60 ns to 120 ns, four input pixels are sent with `VidOrig_nVideoInv = 1`:

$$0x0A,\ 0x32,\ 0xC8,\ 0xFF.$$

The DUT computes the inverted values:

$$\text{out} = 255 - \text{in} \implies 0xF5,\ 0xCD,\ 0x37,\ 0x00.$$

The waveform confirms:

- Correct pixel inversion,
- One-cycle latency between input (`s_tdata`) and output (`m_tdata`),
- Correct forwarding of `tuser` and `tlast`.

Test 2: Inversion Disabled After 130 ns, inversion is disabled (`VidOrig_nVideoInv = 0`) and pixels

$$0x0F,\ 0x80,\ 0xFF$$

are sent. The output matches the input exactly (`m_tdata = s_tdata`).

Conclusion The simulation confirms that:

- AXI4-Stream handshake (`tvalid`, `tready`) is correctly handled,
- Pixel inversion occurs accurately when enabled,
- `tuser` and `tlast` propagate with one-cycle latency,
- Pixel data passes unchanged when inversion is disabled.

The `video_inverter` module thus behaves as expected.

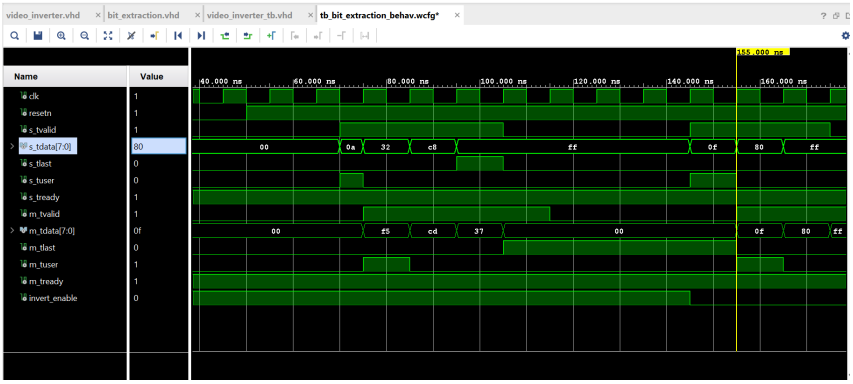


Figure 6.6: TB video inverter VHDL

Chapter 7

Problems and Solutions

Problem	**Impact**	**Proposed Solution / Improvement**
Software version inconsistency	Using different versions of tools or IP cores during development can lead to incompatibilities, unexpected behavior, or synthesis/simulation failures.	Maintain a single, consistent version of all tools and IPs throughout the project. Keep track of version numbers and avoid uncontrolled upgrades. Consider using version control for IP packages.
Final test failures due to IP upgrades	Upgrading an IP core during later stages can break the integration and cause unexpected results in simulation or on hardware.	Develop comprehensive testbenches for each IP module to validate their functionality independently. Automate regression tests to catch issues introduced by upgrades. Where possible, freeze IP versions once they are validated.
Timing violations and interface mismatches	Incorrect synchronization between video modules or clock domains can result in corrupted video output.	Use timing-aware simulations and static timing analysis to verify all clock domains. Insert pipeline registers or FIFOs at AXI4-Stream interfaces to handle latency and maintain throughput.
Limited debugging visibility	Without proper monitoring, it is difficult to identify where data corruption occurs in the video chain.	Integrate debug modules (ILA) at critical points in the video pipeline. Log intermediate video signals and protocol handshakes for analysis.

Table 7.1: Challenges and Improvements in the Video Chain Project

Chapter 8

Conclusion

This project aimed to strengthen our ability to design video-streaming systems while taking into account timing constraints, accuracy requirements, and resource limitations. It challenged us to synthesize our own modules and integrate them into a fully functional system that operates efficiently within realistic constraints.

Additionally, the project emphasized teamwork, resource management, and time management, all of which were essential to achieving meaningful results. Although not all tests were fully completed, the experience greatly expanded our understanding of VHDL design, Vitis HLS development, and system-level integration. We also gained practical knowledge of fundamental IPs such as AXI-Stream modules, video timing controllers, slice components, and binary counters.

Overall, this work provided us with a strong technical foundation and broadened our perspective, preparing us for more advanced and complex digital design projects in the future.

Appendix A

appendix

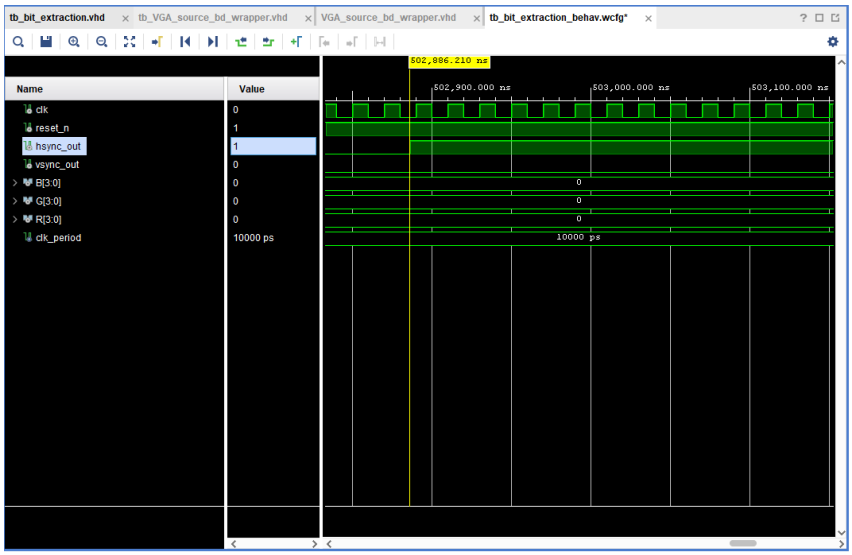


Figure A.1: Bit extraction test to show Vsync and Hsync synchronisation

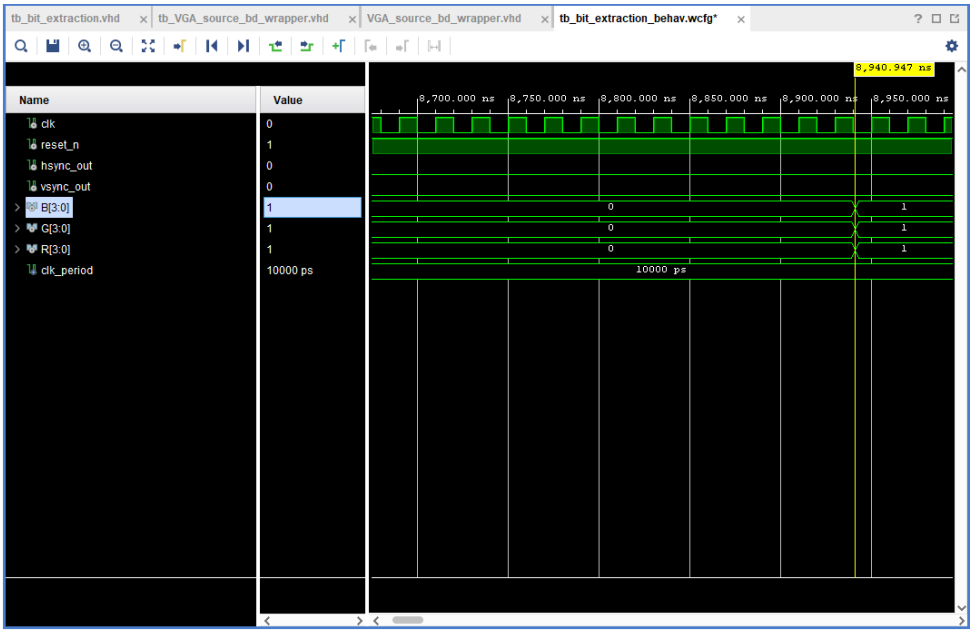


Figure A.2: Test showing each colour synchronisation

```
19  ----------------------------------------------------------------------------------
20
21  library IEEE;
22  use IEEE.STD_LOGIC_1164.ALL;
23
24  entity bit_extraction is
25    Port (
26      Din          : in  std_logic_vector(7 downto 0);
27      Sel_ActVideo : in  std_logic;
28      Dout         : out std_logic_vector(3 downto 0)
29    );
30  end bit_extraction;
31
32  architecture Behavioral of bit_extraction is
33  begin
34    process(Din, Sel_ActVideo)
35    begin
36      if Sel_ActVideo = '1' then
37        Dout <= Din(7 downto 4);
38      else
39        Dout <= (others => '0');
40      end if;
41    end process;
42  end Behavioral;
43
44
```

Figure A.3: Bit extraction VHDL code